

Serena Code Intelligence Guide for OpenCode

Complete Guide to Using OpenCode's Built-in Code Intelligence System

Important: Serena is **NOT an MCP server**. It is OpenCode's built-in code intelligence system that provides advanced semantic code understanding, navigation, and editing capabilities.

Table of Contents

- 1. [What is Serena?](#)
- 2. [Prerequisites](#)
- 3. [Serena Capabilities](#)
- 4. [Getting Started with Serena](#)
- 5. [Configuration](#)
- 6. [Using Serena Tools](#)
- 7. [Advanced Features](#)
- 8. [Best Practices](#)
- 9. [Troubleshooting](#)
- 10. [Reference](#)
- 11. [FAQ](#)

What is Serena?

Serena is OpenCode's integrated code intelligence system that provides:

- ✔ **Semantic Code Understanding** - Understands code structure, not just text
- ✔ **Symbol Navigation** - Jump to definitions, find implementations, locate references
- ✔ **LSP Integration** - Language Server Protocol support for multiple languages
- ✔ **Smart Code Editing** - Symbol-aware editing and refactoring
- ✔ **Diagnostics** - Real-time code analysis and error detection
- ✔ **Project Memory** - Persistent context and learning about your codebase
- ✔ **Efficient Context Management** - Token-efficient code exploration

Why Serena?

Traditional code tools work with **text** and **lines**. Serena works with **symbols** and **structure**.

Example:

- **Traditional tool:** "Read lines 100-200 of file.py"
- **Serena:** "Get me the `processUser` method in the `UserService` class"

This makes Serena:

- **More efficient** - Lower token usage
- **More precise** - Exact code targeting
- **More intelligent** - Understands relationships between code
- **More reliable** - Less prone to errors from line number changes

Key Difference from MCP Servers

Aspect	Serena	MCP Servers
--------	--------	-------------

Type	Built-in code intelligence	External tool integrations
Installation	Already included in OpenCode	Must be installed separately
Purpose	Code understanding & editing	External service access
Activation	Automatic in coding contexts	Manual configuration required
Examples	Symbol search, refactoring	GitHub, Sentry, databases

Prerequisites

System Requirements

Serena is built into OpenCode. You need:

1. **OpenCode** installed and working
 - Version: 1.3.0 or higher recommended
 - Installation guide: See main OpenCode installation documentation
2. **LSP Support** (Optional but recommended)
 - Enables enhanced code intelligence
 - Configure in `opencode.json`
3. **Project Structure**
 - Git repository (recommended)
 - Supported languages (see Language Support section)

Supported Languages

Serena provides varying levels of support for different languages:

Language	Support Level	Features
TypeScript	★★★★★ Full	All features
JavaScript	★★★★★ Full	All features
Python	★★★★★ Full	All features
Java	★★★★ High	Most features
C/C++	★★★★ High	Most features
Go	★★★★ High	Most features
Rust	★★★ Medium	Basic features
PHP	★★★ Medium	Basic features
Ruby	★★★ Medium	Basic features

Serena Capabilities

1. Code Navigation

Find Declarations

```
Find where `processPayment` function is defined
```

Serena will:

- Locate the exact definition
- Show file location
- Display function signature
- Include documentation if available

Find Implementations

```
Show all classes that implement `PaymentProcessor` interface
```

Serena will:

- Find all implementing classes
- Show implementation details
- Display file locations
- Include inheritance hierarchy

Find References

```
Find all places where `UserService.authenticate` is called
```

Serena will:

- Locate all usages
- Show code context
- Identify reference types (call, import, etc.)
- Provide file and line information

2. Symbol Management

Symbol Search

```
Find all methods named `validate` in the project
```

Symbol Overview

```
Show me an overview of the `UserController` class
```

Output includes:

- Class structure
- Method signatures
- Properties
- Nested classes

Symbol Hierarchy

Show the class hierarchy for ``BaseRepository``

3. Code Editing

Symbol Body Replacement

Replace the body of the ``calculateTotal`` method with optimized version

Insert Before/After Symbol

Add a new ``validateInput`` method before ``processData`` in `UserService`

Content Replacement

Replace all occurrences of deprecated API calls with new ones

4. Code Analysis

Diagnostics

Check for errors and warnings in `authentication.ts`

Output includes:

- Syntax errors
- Type errors
- Linting warnings
- Code smells

Symbol Relationships

Show what components depend on `UserRepository`

5. Project Memory

Serena maintains a persistent memory system:

- **Code Patterns** - Learns coding conventions
- **Architecture** - Understands project structure
- **Context** - Remembers previous work
- **Best Practices** - Notes team standards

Getting Started with Serena

Step 1: Verify Serena is Available

Serena is automatically available in OpenCode when working with code files.

Test Serena:

Show me the structure of this file: `src/services/user-service.ts`

If Serena responds with a structured overview, it's working!

Step 2: Initialize Your Project

1. Navigate to your project:

```
cd /path/to/your/project
opencode
```

2. Initialize OpenCode for the project:

```
/init
```

This creates `AGENTS.md` and helps Serena understand your codebase.

3. Verify LSP is enabled (recommended):

Check `opencode.json` :

```
{
  "lsp": true
}
```

Step 3: First Serena Query

Try these starter queries:

Get Project Overview:

Give me an overview of the main entry point file

Find a Specific Symbol:

Find the User class definition

Understand Code Structure:

Show me all public methods in the AuthService class

Step 4: Using Serena for Editing

Ask Serena to make changes:

Refactor the validateEmail method to use regex instead of string operations

Serena will:

1. Find the method definition
 2. Read the current implementation
 3. Generate improved code
 4. Replace the method body
 5. Verify the change
-

Configuration

Basic Configuration

Serena works automatically, but you can enhance it with configuration.

Enable LSP (Recommended)

Location: `~/.config/opencode/opencode.json` or project `opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true
}
```

Benefits:

- Better code intelligence
- More accurate symbol resolution
- Enhanced diagnostics
- Faster symbol search

Customize LSP Servers

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": {
    "typescript": {
      "disabled": false
    },
    "python": {
      "disabled": false
    },
    "java": {
      "disabled": false
    }
  }
}
```

Project-Specific Configuration

Create `opencode.json` in your project root:

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true,
  "instructions": [
    "CONTRIBUTING.md",
    "docs/architecture.md"
  ],
  "watcher": {
```

```
"ignore": [
  "node_modules/**",
  "dist/**",
  "build/**",
  ".git/**"
]
}
```

Memory Configuration

Serena's memory system stores project knowledge.

Memory Locations:

- Global: `~/.config/opencode/memories/`
- Project: `<project>/.opencode/memories/`

Managing Memories:

```
/memories list
/memories read architecture/overview
/memories delete old-pattern
```

Performance Configuration

For large projects:

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true,
  "snapshot": true,
  "watcher": {
    "ignore": [
      "node_modules/**",
      "vendor/**",
      "*.log",
      ".git/**"
    ]
  },
  "compaction": {
    "auto": true,
    "prune": true,
    "reserved": 10000
  }
}
```

Using Serena Tools

Serena provides specialized tools for code intelligence. OpenCode uses these automatically, but understanding them helps you work more effectively.

1. Symbol Overview

Purpose: Get a structural overview of a file without reading all content.

When to use:

- First time exploring a file
- Understanding class/module structure
- Planning edits

Example interaction:

```
Show me the structure of src/controllers/payment.controller.ts
```

Serena provides:

- Class names
- Method signatures
- Properties
- Imports/exports

2. Find Symbol

Purpose: Search for symbols by name or pattern.

When to use:

- Finding specific functions/classes
- Locating implementations
- Cross-file symbol search

Example interactions:

```
Find the authenticate method in UserService  
Find all classes that end with "Repository"  
Find functions starting with "process"
```

Serena returns:

- Symbol locations
- Signatures
- Documentation
- Optionally, full body

3. Find Declaration

Purpose: Jump to where something is defined.

When to use:

- Following imports
- Understanding symbol origins
- Tracing dependencies

Example interactions:


```
Where is PaymentProcessor defined?  
Show me the declaration of the Config interface
```

4. Find Implementations

Purpose: Find all implementations of an interface or abstract class.

When to use:

- Understanding inheritance
- Finding concrete implementations
- Refactoring abstractions

Example interactions:

```
Show all implementations of IUserRepository  
Find classes that extend BaseService
```

5. Find References

Purpose: Find everywhere a symbol is used.

When to use:

- Impact analysis before changes
- Understanding dependencies
- Refactoring safely

Example interactions:

```
Find all uses of the deprecated oldApiCall function  
Show where UserService.authenticate is called
```

6. Get Diagnostics

Purpose: Check for errors, warnings, and code issues.

When to use:

- Before committing
- After refactoring
- Debugging

Example interactions:

```
Check for errors in the authentication module  
Show warnings in payment-processor.ts
```

7. Symbol Editing

Replace Symbol Body:

```
Refactor the calculateDiscount method to use the new pricing rules
```

Insert Before/After:

```
Add input validation before the processOrder method
```

Content Replacement:

```
Update all Logger.log calls to use the new Logger.info method
```

Advanced Features

1. Intelligent Code Refactoring

Serena understands code relationships, enabling safe refactoring.

Example: Rename with references

```
Rename the UserService class to UserManager and update all references
```

Serena will:

1. Find the class definition
2. Locate all references
3. Rename the symbol
4. Update all import statements
5. Update all usages

Example: Extract method

```
Extract the validation logic from processPayment into a new validatePaymentData method
```

2. Cross-Language Support

Serena can work across multiple languages in the same project.

Example: Polyglot project

```
Find all API endpoints defined in:  
- TypeScript controllers  
- Python Flask routes  
- Java Spring controllers
```

3. Project Learning

Serena learns from your project over time.

What Serena learns:

- Common patterns
- Naming conventions
- Architecture decisions
- Team practices

Using learned context:

```
Create a new service following the existing pattern
```

```
Add a new API endpoint like the user endpoints
```

4. Symbol-Based Edits

Serena can perform precise, symbol-aware edits.

Example: Update all methods in a class

```
Add error handling to all public methods in UserService
```

Example: Consistent changes

```
Update all repository classes to use the new database connection pattern
```

5. Memory System

Write to memory:

```
Remember that we use camelCase for method names and PascalCase for classes
```

Read from memory:

```
What's our authentication pattern?
```

List memories:

```
Show all memories about API design
```

Best Practices

1. Let Serena Guide Your Navigation

✗ Don't:

```
Read the entire user-service.ts file
```

✓ Do:

```
Show me the structure of user-service.ts first  
Then show me just the authenticate method
```

Why: More efficient, better context management, lower token usage.

2. Use Symbol-Based Editing

✗ Don't:

```
Read lines 100-150, then replace them with new code
```

✓ Do:

```
Replace the processPayment method with the new implementation
```

Why: More reliable, survives line number changes, respects code structure.

3. Check Impact Before Refactoring

✅ **Always:**

```
Find all references to oldFunction before renaming it
```

Why: Prevents breaking changes, identifies dependencies.

4. Use Diagnostics Regularly

✅ **Good habit:**

```
Check for errors after making changes  
Show diagnostics for files I just edited
```

Why: Catch issues early, maintain code quality.

5. Leverage Project Memory

✅ **Document as you go:**

```
Remember that the payment module uses Stripe API v2023-10-16  
Remember our error handling pattern: try-catch with specific error types
```

Why: Builds project knowledge, helps future work.

6. Start Broad, Then Narrow

✅ **Effective approach:**

1. Get file overview
2. Find relevant symbol
3. Read specific symbol body
4. Make targeted edit

❌ **Inefficient approach:**

1. Read entire file
2. Search manually
3. Edit by line numbers

7. Use Serena for Discovery

✅ **Examples:**

```
Find all functions that interact with the database  
Show me the authentication flow  
What classes use the Logger?
```

Troubleshooting

Issue 1: Serena Not Responding

Symptoms: Serena queries don't return results or show errors.

Diagnosis:

```
/init  
Check if project is initialized
```

Solutions:

1. **Reinitialize project:**

```
/init
```

2. **Check LSP status:**

```
{  
  "lsp": true  
}
```

3. **Verify file type is supported:** Make sure you're working with code files (.ts, .js, .py, etc.)

4. **Check working directory:**

```
pwd  
# Should be in project root or subdirectory
```

Issue 2: Symbol Not Found

Symptoms: "Symbol not found" or empty results.

Solutions:

1. **Use broader search:**

```
# Instead of exact match  
Find the authenticate method  
  
# Try pattern  
Find methods with "auth" in the name
```

2. **Check symbol path:**

```
# If searching in UserService/authenticate  
# Try just authenticate first
```

3. **Verify file is saved:** Unsaved changes might not be indexed yet.

4. **Check file patterns:**

```
{  
  "watcher": {  
    "ignore": ["node_modules/**"]  
  }  
}
```

```
}  
}
```

Issue 3: Slow Symbol Search

Symptoms: Symbol operations take a long time.

Solutions:

1. **Enable LSP** (if not already):

```
{  
  "lsp": true  
}
```

2. **Exclude large directories:**

```
{  
  "watcher": {  
    "ignore": [  
      "node_modules/**",  
      "vendor/**",  
      "dist/**",  
      "build/**",  
      ".git/**"  
    ]  
  }  
}
```

3. **Restart OpenCode:**

```
# Close and reopen  
opencode
```

4. **Clear cache** (if persistent):

```
rm -rf ~/.local/share/opencode/cache/
```

Issue 4: Incorrect Symbol Information

Symptoms: Serena returns outdated or incorrect symbol info.

Solutions:

1. **Save all files:** Unsaved changes aren't indexed.
2. **Reinitialize project:**

```
/init
```

3. **Check for syntax errors:**

```
Show diagnostics for this file
```

4. **Restart LSP** (if enabled):

```
# Restart OpenCode  
opencode
```

Issue 5: Edit Conflicts

Symptoms: Serena edit fails or produces unexpected results.

Solutions:

1. **Verify symbol exists:**

```
Show me the structure of the file first
```

2. **Use exact symbol name:**

```
# Instead of  
Edit the login method  
  
# Use  
Edit the authenticateUser method in AuthService
```

3. **Check for multiple symbols:**

```
Find all methods named processData
```

4. **Use regex mode for complex edits:**

```
Replace content in file.ts using regex pattern
```

Issue 6: Memory Issues

Symptoms: Serena uses too much memory or crashes.

Solutions:

1. **Enable compaction:**

```
{  
  "compaction": {  
    "auto": true,  
    "prune": true  
  }  
}
```

2. **Reduce context window:** Start new conversation: `/new`

3. **Disable snapshot for large repos:**

```
{
  "snapshot": false
}
```

4. Increase swap space (Linux/macOS):

```
# Check current swap
swapon --show
```

Issue 7: LSP Errors

Symptoms: LSP-related errors or warnings.

Solutions:

1. Check LSP configuration:

```
{
  "lsp": {
    "typescript": {
      "disabled": false
    }
  }
}
```

2. Install language servers:

```
# TypeScript
npm install -g typescript typescript-language-server

# Python
pip install python-lsp-server

# Go
go install golang.org/x/tools/gopls@latest
```

3. Check PATH:

```
# Ensure language servers are in PATH
which typescript-language-server
which pylsp
which gopls
```

4. Disable problematic LSP:

```
{
  "lsp": {
    "problematic-language": {
      "disabled": true
    }
  }
}
```



```
}  
}  
}
```

Issue 8: Permission Denied

Symptoms: Serena can't read/write files.

Solutions:

1. Check file permissions:

```
ls -la file.ts
```

2. Fix permissions:

```
chmod 644 file.ts
```

3. Check directory permissions:

```
ls -la .  
chmod 755 .
```

4. Run from correct directory:

```
cd /path/to/project  
opencode
```

Debugging Tools

Enable debug mode:

```
export DEBUG=opencode:*  
opencode
```

Check Serena logs:

```
tail -f ~/.local/share/opencode/logs/opencode.log
```

Test Serena functionality:

```
/init  
Show me the structure of any .ts file
```

Reference

Symbol Name Paths

Serena uses **name paths** to identify symbols uniquely.

Format: ParentSymbol/ChildSymbol/GrandchildSymbol

Examples:

Code	Name Path
class UserService	UserService
method authenticate in UserService	UserService/authenticate
nested class Config in UserService	UserService/Config
Overloaded methods (Java)	UserService/process[0], UserService/process[1]

Patterns:

Pattern	Matches
authenticate	Any symbol named authenticate
UserService/authenticate	authenticate in UserService
/UserService/authenticate	Exact path from file root
*Service	Symbols ending with "Service"
get*	Symbols starting with "get"

LSP Symbol Kinds

Serena uses LSP symbol kinds for categorization:

Kind	Number	Description
File	1	File
Module	2	Module/namespace
Namespace	3	Namespace
Package	4	Package
Class	5	Class
Method	6	Method
Property	7	Property
Field	8	Field
Constructor	9	Constructor
Enum	10	Enumeration
Interface	11	Interface

Function	12	Function
Variable	13	Variable
Constant	14	Constant
String	15	String
Number	16	Number
Boolean	17	Boolean
Array	18	Array

Filter by kind:

Find all classes (kind 5) in the project
Find all methods (kind 6) in UserService

Diagnostic Severity Levels

Level	Number	Description
Error	1	Must be fixed
Warning	2	Should be fixed
Information	3	FYI
Hint	4	Suggestion

Filter diagnostics:

Show only errors (severity 1) in auth.ts
Show warnings and errors in payment module

Tool Overview

Tool	Purpose	Key Parameters
get_symbols_overview	File structure	relative_path, depth
find_symbol	Search symbols	name_path_pattern, include_body
find_declaration	Jump to definition	relative_path, regex
find_implementations	Find implementations	name_path, relative_path
find_referencing_symbols	Find usages	name_path, relative_path
get_diagnostics_for_file	Check errors	relative_path, min_severity
replace_symbol_body	Edit symbol	name_path, body
insert_before_symbol	Insert before	name_path, body

insert_after_symbol	Insert after	name_path, body
replace_content	Regex replace	relative_path, needle, repl
rename_symbol	Rename symbol	name_path, new_name
safe_delete_symbol	Delete symbol	name_path_pattern

Memory Commands

Command	Purpose
/memories list	List all memories
/memories list topic	List memories by topic
/memories read name	Read specific memory
/memories write name content	Write memory
/memories delete name	Delete memory

Configuration Options

Option	Type	Default	Description
lsp	boolean/object	false	Enable LSP
snapshot	boolean	true	Enable undo/redo
watcher.ignore	array	[]	Ignore patterns
compaction.auto	boolean	true	Auto-compact context
compaction.prune	boolean	true	Prune old outputs

FAQ

Q: Is Serena an MCP server?

A: No. Serena is OpenCode's built-in code intelligence system. It's not an MCP server and doesn't require installation or configuration like MCP servers do.

Q: Do I need to install anything for Serena?

A: No. Serena comes built-in with OpenCode. However, enabling LSP (`"lsp": true`) will enhance its capabilities.

Q: Which languages does Serena support?

A: Serena supports TypeScript, JavaScript, Python, Java, C/C++, Go, Rust, PHP, Ruby, and more. Support level varies by language (see Language Support section).

Q: How does Serena differ from traditional code search?

A: Traditional search works with text and line numbers. Serena understands code structure (classes, methods, etc.) and relationships, making it more precise and efficient.

Q: Can Serena work with large codebases?

A: Yes. Serena is designed for large projects. Use the `watcher.ignore` configuration to exclude large directories like `node_modules`.

Q: Does Serena require internet connection?

A: No. Serena works entirely locally.

Q: Can I use Serena with non-Git projects?

A: Yes, but Git repositories provide better context and history tracking.

Q: How do I know if Serena is working?

A: Try: `Show me the structure of [any code file]`. If you get a structured response, Serena is working.

Q: Can Serena edit multiple files at once?

A: Yes. Serena can make changes across multiple files, especially when updating references.

Q: What's the difference between Serena and LSP?

A: LSP provides code intelligence to Serena. Enabling LSP makes Serena more powerful, but Serena can work without it.

Q: Can I disable Serena?

A: Serena is integral to OpenCode's code intelligence. You can disable LSP (`"lsp": false`), but Serena itself is always available.

Q: How does Serena handle syntax errors?

A: Serena can still provide symbol information even with syntax errors, though some features may be limited until errors are fixed.

Q: Can Serena work with proprietary/closed-source code?

A: Yes. All processing happens locally on your machine.

Q: Does Serena learn from my code?

A: Yes. Serena's memory system learns patterns, conventions, and architectural decisions from your project.

Q: How do I reset Serena's project knowledge?

A: Delete project memories: `/memories delete [name]` or reinitialize: `/init`

Q: Can Serena work across multiple projects?

A: Serena works project-by-project. Switch projects by changing directories and running `/init`.

Example Workflows

Workflow 1: Understanding a New Codebase

1. **Initialize project:**

```
/init
```

2. **Get high-level overview:**

```
Show me the main entry point file structure  
What are the main modules in this project?
```

3. **Explore key components:**

```
Show me the structure of src/services/user-service.ts  
Find all classes that end with "Controller"
```

4. **Understand relationships:**

```
What depends on UserService?  
Find all implementations of IRepository
```

5. **Save insights:**

```
Remember that the project uses a layered architecture: Controllers → Services →  
Repositories
```

Workflow 2: Refactoring a Module

1. **Analyze current state:**

```
Show me all methods in UserService  
Find all references to UserService.authenticate
```

2. **Check for issues:**

```
Show diagnostics for user-service.ts
```

3. **Make changes:**

```
Refactor the authenticate method to use async/await instead of callbacks
```

4. **Verify impact:**

```
Find all files that import UserService  
Check diagnostics after the change
```

5. **Update tests:**

```
Update the test file to match the new authenticate signature
```

Workflow 3: Adding a New Feature

1. Understand existing patterns:

```
Show me how other services are structured
What's the pattern for API endpoints?
```

2. Create new code:

```
Create a new PaymentService following the existing service pattern
Add methods for processPayment and refundPayment
```

3. Integrate:

```
Add PaymentService to the dependency injection container
Create a new PaymentController
```

4. Test:

```
Check diagnostics for the new files
Verify no import errors
```

Workflow 4: Debugging

1. Locate issue:

```
Show diagnostics for authentication module
Find the authenticateUser method
```

2. Understand context:

```
Find all references to authenticateUser
Show me the User class definition
```

3. Analyze:

```
Check for errors in the authentication flow
```

4. Fix:

```
Update authenticateUser to handle null user case
```

5. Verify:

```
Check diagnostics after fix
Find references to ensure no breaking changes
```

Comparison with Other Tools

Serena vs. Traditional Grep/Find

Feature	Serena	Grep/Find
Understands code	✔ Yes	✗ No (text only)
Symbol-aware	✔ Yes	✗ No
Handles refactoring	✔ Yes	✗ No
Cross-references	✔ Yes	⚠ Limited
Type information	✔ Yes	✗ No
Line number independent	✔ Yes	✗ No
Token efficient	✔ Yes	⚠ Can be verbose

Serena vs. IDE Features

Feature	Serena	IDE
Works in terminal	✔ Yes	✗ No
AI-integrated	✔ Yes	⚠ Plugin-based
Cross-file refactoring	✔ Yes	✔ Yes
Natural language queries	✔ Yes	✗ No
Context-aware	✔ Yes	⚠ Limited
Learns project patterns	✔ Yes	✗ No

Serena vs. Language Servers (LSP)

Aspect	Serena	LSP
Relationship	Uses LSP as data source	Provides data
Scope	Project-wide intelligence	File/symbol info
Integration	Built into OpenCode	External servers
Query interface	Natural language	Programmatic
Editing	Symbol-aware edits	Provides info only

Appendix A: Quick Reference Card

Common Queries

```
# Navigation
Show file structure
Find [symbol name]
Where is [symbol] defined?
Show all methods in [class]
```



```
# Analysis
Check errors in [file]
Find references to [symbol]
Show implementations of [interface]
What depends on [module]?

# Editing
Refactor [method] to [description]
Rename [old_name] to [new_name]
Add [new code] before [symbol]
Replace [symbol] with [new implementation]

# Memory
Remember [important pattern]
What's the pattern for [aspect]?
List memories about [topic]
```

Configuration Essentials

```
{
  "$schema": "https://opencode.ai/config.json",
  "lsp": true,
  "snapshot": true,
  "watcher": {
    "ignore": ["node_modules/**", "dist/**"]
  }
}
```

Keyboard Shortcuts

Action	Command
Initialize	/init
New session	/new
List memories	/memories list
Help	/help

Appendix B: Language-Specific Notes

TypeScript/JavaScript

Optimal setup:

```
{
  "lsp": {
    "typescript": {
      "disabled": false
    }
  }
}
```

```
}  
}  
}
```

Best practices:

- Keep `tsconfig.json` up to date
- Use explicit types for better symbol resolution
- Organize imports for clearer structure

Python

Optimal setup:

```
{  
  "lsp": {  
    "python": {  
      "disabled": false  
    }  
  }  
}
```

Install LSP server:

```
pip install python-lsp-server
```

Best practices:

- Use type hints for better analysis
- Follow PEP 8 for consistent structure
- Document with docstrings

Java

Optimal setup:

```
{  
  "lsp": {  
    "java": {  
      "disabled": false  
    }  
  }  
}
```

Best practices:

- Keep Maven/Gradle configs updated
- Use package structure consistently
- Handle overloaded methods with indices

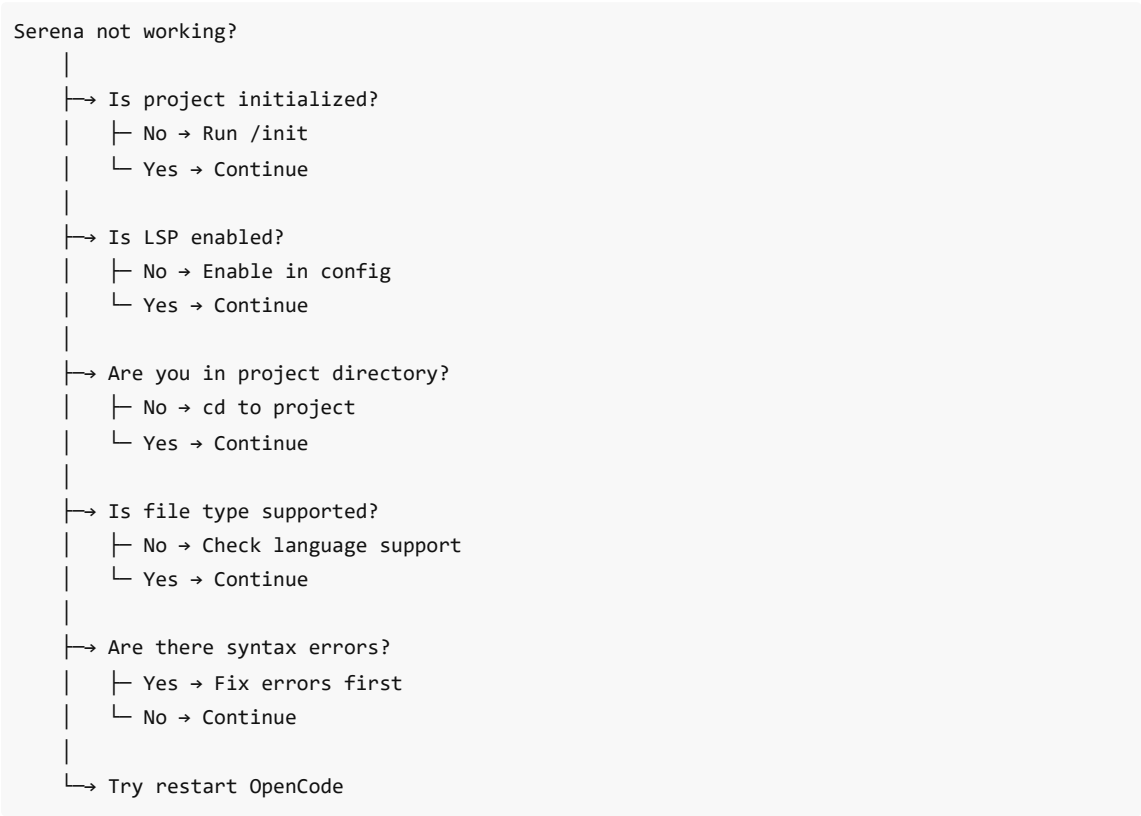
C/C++

Note: May require additional setup for best results.

Best practices:

- Keep compile_commands.json updated
- Use consistent header guards
- Document complex macros

Appendix C: Troubleshooting Flowchart



Glossary

Term	Definition
Symbol	A named code entity (class, function, method, variable, etc.)
Name Path	Hierarchical path identifying a symbol (e.g., Class/method)
LSP	Language Server Protocol - provides code intelligence
Serena	OpenCode's built-in code intelligence system
Diagnostic	Error, warning, or hint about code
Reference	Location where a symbol is used
Implementation	Concrete realization of an interface or abstract class
Declaration	Where a symbol is defined

Memory	Persistent knowledge Serena stores about projects
Snapshot	Git-based system for undo/redo functionality

Document Information:

- **Version:** 1.0
- **Last Updated:** May 15, 2026
- **OpenCode Compatibility:** 1.3.0+
- **Author:** OpenCode Documentation Team

Note: Serena is continuously evolving. Check the official OpenCode documentation for the latest features and updates.

End of Document